



Softwarearchitektur Projektarbeit

Gen-AI Nachweis

Gruppe 1 - Team 2

Ayleen Edenhauser, Matteo Volperino,
Jakob Oberhofer, Leonid Georg Mikhailovic Stommel

WS 2025/26

Einleitung

Die Arbeit mit Künstlicher Intelligenz (KI) und Large Language Models (LLMs) wird immer wichtiger und unumgänglicher. Damit wird es auch immer wichtiger zu lernen wie man verantwortungsvoll mit KI umgeht. Im Optimalfall sollte man nach einem gewissenhaften Prompt in der Softwareentwicklung nicht nur einen schlaueren Code sondern auch einen schlaueren Kopf haben, so dass man es als Hilfsmittel für ein besseres Verständnis für ein Thema einsetzt.

In diesem Dokument dokumentieren wir relevante Prompts, welche uns in der Softwarearchitektur Projektarbeit weitergeholfen haben.

Contents

1	Ayleen Edenhauser	4
1.1	Unterstützung bei APIs/Endpoints	4
1.1.1	Kernidee	4
1.1.2	Beispiel zum Verständnis	4
1.1.3	Lernergebnisse	4
1.2	Weitere Nutzung	5
2	Matteo Volperino	6
3	Jakob Oberhofer	7
3.1	Unit-Tests für Geschäftslogik (Testfallideen)	7
3.1.1	Prompt	7
3.1.2	Lernergebnis	7
3.2	Analyse von Fehlercodes	7
3.2.1	Prompt	7
3.2.2	Lernergebnis	8
4	Leonid Georg Mikhailovic Stommel	9
4.1	Library-Problem bei API-UnitTests	9
4.1.1	Prompting	9
4.1.2	Lernergebnisse	9
4.2	Generierung von Dummy-Produkten	10
4.2.1	Prompting	10
4.2.2	Lernergebnisse	10

1 Ayleen Edenhauser

1.1 Unterstützung bei APIs/Endpoints

1.1.1 Kernidee

I'm working on a project. The task is to create a web shop. I'm responsible for the shopping cart. We're using Java + Spring in the backend, Typescript + React in the frontend. The shopping cart is saved only in the browser's local storage. I've implemented basic functions such as `getCart`, `addToCart` and `removeFromCart`. Now I'm working on the API, but the cart only exists in the frontend. How can I tackle this? Usually, there are controllers in the backend, mapping HTTP requests to methods in the backend, but the cart doesn't exist there. What can I do? Do not show me any code.

Antwort

[Link zum Chat.](#)

1.1.2 Beispiel zum Verständnis

Can you give me a code example of how these endpoints could look?

Antwort

[Link zum Chat.](#)

1.1.3 Lernergebnisse

Daraus habe ich gelernt:

- Der Warenkorb wird im Backend als **Input** verwendet
- Der Warenkorb arbeitet sehr eng mit **Bestellungen** zusammen
- Eventuell wird ein `CartController` im Backend benötigt

1.2 Weitere Nutzung

Die Ausarbeitung des Backends war relativ simpel, deshalb habe ich GenAI des weiteren nur dazu verwendet, States und Effects in TypeScript besser zu verstehen. Allgemein war im Frontend die [React Dokumentation](#) eine weitaus bessere Hilfe als GenAI.

2 Matteo Volperino

Ich habe GenAI hauptsächlich für TypeScript genutzt, sei es für kleinere syntaktische Fragen oder für die Umsetzung von UI-Design-Aspekten. Dadurch habe ich Flexbox kennengelernt und diese Technik zur Gestaltung der Produktseite eingesetzt. Insgesamt habe ich GenAI eher sparsam verwendet. Beim Debugging von API-Problemen war die KI größtenteils hilfreich. Bei einem Forbidden-Fehler konnte sie jedoch keine konkrete Lösung liefern und drehte sich teilweise im Kreis. Das zugrunde liegende Problem wurde schließlich von unserer CTO, Ayleen Edenhauser, identifiziert. Es handelte sich um einen Autorisierungsfehler in einer User-Funktion, die @PreAuthorize und User mit der Customer-Rolle nicht ausführen konnten. Wir wollten von Anfang an Bilder anzeigen, waren uns jedoch nicht sicher, wie dies am effizientesten umgesetzt werden könnte. ChatGPT schlug vor, die Bilder entweder als BLOBs direkt in der Datenbank zu speichern oder die URLs der Bilder separat abzulegen. Wir entschieden uns für Letzteres und nutzten dabei Links zu Bildern aus dem Internet.

3 Jakob Oberhofer

Im Rahmen der Entwicklung des Order-Moduls und des User Signups habe ich KI gezielt zur Unterstützung eingesetzt. Der Fokus lag dabei auf der Fehleranalyse, sowie auf der effizienten Erstellung von Testfällen.

3.1 Unit-Tests für Geschäftslogik (Testfallideen)

3.1.1 Prompt

Erstelle mir einen JUnit-Test für den `OrderService`. Der Test soll sicherstellen, dass eine `InsufficientStockException` geworfen wird, wenn ein Benutzer mehr Produkte bestellt, als im Lager verfügbar sind.

LLM-Ausgabe

<Codeteil für den Testfall.>

3.1.2 Lernergebnis

Mit weiteren ähnlichen Prompts wie diesem konnte ich mir durch die Verwendung von KI einiges an Schreibarbeit ersparen. Die KI lieferte schnell das Mockito-Grundgerüst. Ich musste jedoch manuell nachbessern. Außerdem konnte mir die KI dabei helfen weitere Testfälle abzudecken auf die ich vielleicht nicht gedacht hätte.

3.2 Analyse von Fehlercodes

3.2.1 Prompt

Ich erhalte beim ausführen der Tests/starten des Backends folgenden Error:...

LLM-Ausgabe

<Erklärung des Fehlercodes oder Hinweise wo der Fehlerhafte Codeteil liegen könnte.>

3.2.2 Lernergebnis

Die KI half hier, Zeit bei der Fehlersuche zu sparen. Außerdem wurden Fehlermeldungen welche teils unverständlich für mich waren so erklärt, dass ich sie schnell verstehen und beheben konnte.

4 Leonid Georg Mikhailovic Stommel

4.1 Library-Problem bei API-UnitTests

[Link zum Chat.](#)

4.1.1 Prompting

Die Unit-Tests, welche API-Calls für das Feature "Products" testet, schlugen fehl und ich konnte bei den sehr langen Fehlermeldungen nicht durchblicken woran es lag, da ich dort sehr schnell die Übersicht verlor.

Der LLM habe ich nach Erklärung des Softwarekontextes mein Problem beschrieben und sowohl den Source-Code der Datei `ProductController.java`, als auch die Fehlermeldungen für alle fehlgeschlagenen Tests mitgegeben.

LLM-Ausgabe

Die LLM schlug mir vor, den `JwtTokenProvider` als Mock einzubinden. Doch es gab wiederum eine Fehlermeldung, ebenso lang und unübersichtlich. Diese habe ich dann als Antwort geschickt und den Vorschlag bekommen ebenso noch `JwtConfig` einzubinden. Daraufhin hat es funktioniert.

Abschließend habe ich die LLM noch nach einem Fazit der Problemlösung und was ich daraus gedanklich mitnehmen kann gefragt.

4.1.2 Lernergebnisse

Daraus habe ich gelernt:

- Erkennen von Kausalitäten bei bestimmten Statements, z.B. wenn `@WebMvcTest(ProductController.class)` nur einen abgekapselten Teil lädt, bedeutet dies auch, dass andere Komponenten fehlen, die explizit eingebunden werden müssen, sofern sie für den Test benötigt werden.
- Besseres Verständnis des Mocking-Systems (es werden nicht nur eigene Klassen gemockt, sondern auch weniger offensichtliche Abhängigkeiten).
- Sich nicht von jeder Fehlermeldung abschrecken zu lassen und Fehlerausgaben selbst genauer zu analysieren.

4.2 Generierung von Dummy-Produkten

[Link zum Chat.](#)

4.2.1 Prompting

Dummy-Daten sind bereits in der Entwicklung einer Software sinnvoll, damit sich so zukünftige Daten visualisieren lassen, die Software getestet wird und auftretende Fehler entlarvt werden können.

Wir haben uns dazu entschieden Videospiele als Produkte zu nehmen. Für das Einpflegen von Dummy-Daten könnte ich jetzt bei jedem Videospiel alle Metadaten raussuchen, was sehr zeitintensiv wäre und sich je nach Anzahl der Produkte sehr repetitiv und als nicht besonders fordernde Arbeit entlarven würde.

Was ich stattdessen gemacht habe, ist der LLM 100 Spiele generieren zu lassen. Dabei hatte ich einige Favoriten, wo ich die Titel genannt habe, den Rest habe ich der LLM überlassen.

LLM-Ausgabe

Das generieren der Titel und Beschreibungen und auch passende Lageranzahl war sehr zufriedenstellend. Das Problem waren die Bilder. Da ich nicht jedes Spiel-Artwork für's testen auf den Server laden wollte, wollte ich, dass die LLM auf öffentliche Ressourcen zugreift. Die Ausgabe sah auch sehr vielversprechend aus, da es Links von wikimedia waren (frei zugänglich). Doch waren davon viele Links ungültig und es konnte kein Bild angezeigt werden. Außerdem wollte die LLM anfänglich ein eigenes Fallback implementieren, bei dem ohne Bild lediglich der Name des Spieletitels angezeigt wird. Jedoch war dies überflüssig, da ich bereits ein Sample-Bild als Fallback für ein fehlendes Bild implementiert habe.

4.2.2 Lernergebnisse

Daraus habe ich gelernt:

- LLMs können gut schriftliche Dummy-Informationen generieren.
- LLMs generieren oftmals Weblinks, die überhaupt nicht existieren.
- LLMs machen oftmals zu viel, was oftmals nicht gut ist.